РУКОВОДСТВО ДЛЯ РАЗРАБОТЧИКА · 28.10.2026

Приём DASH и подключение кошелька

DashPayBox + Dash Platform Extension

1. Архитектура интеграции

Для подключения оплаты нужны два компонента: микросервис DashPayBox на стороне магазина и расширение Dash Platform Extension у покупателя. Backend создаёт счёт, сайт вызывает requestSendPayment с адресом и суммой, кошелёк выполняет перевод. DashPayBox обнаруживает платёж и подтверждает заказ через webhook.

Компонент	Задача
Frontend магазина	Показывает счёт, QR-код, срок оплаты и статус заказа. Предлагает оплату через расширение при наличии requestSendPayment, а также QR-код и dash: URI.
Backend магазина	Определяет сумму заказа, создаёт счёт через API DashPayBox, проверяет webhook и выдаёт товар или доступ.
DashPayBox	Создаёт одноразовый адрес, фиксирует курс fiat → DASH, отслеживает платёж и переводит средства на адрес продавца.
Dash Platform Extension	Планируемый requestSendPayment показывает запрос оплаты, получает подтверждение покупателя и выполняет перевод. Приватные ключи остаются в кошельке.

Поток оплаты

Покупатель → checkout → backend магазина → DashPayBox → адрес и сумма → requestSendPayment или QR → подтверждение в кошельке → подписанный webhook → заказ оплачен.

Что подключает разработчик магазина

HTTP API DashPayBox на backend, обработчик подписанного webhook и одну кнопку на frontend с вызовом requestSendPayment. Устанавливать Dash SDK на сайт, создавать State Transition или реализовывать подпись транзакций не требуется: перевод выполняет расширение.

> **Статус DashPayBox:** версия 0.1.0, pre-1.0 / testnet / evaluation-grade. BLS-проверка InstantLock и ChainLock ещё не реализована; финальность определяется по данным DAPI-узла. Перед использованием для реальных платежей требуется устранить это ограничение. Для пилота используйте testnet и доверенные DAPI-узлы.

Инструкция составлена по локальному исходному коду DashPayBox 0.1.0 и Dash Platform Extension 1.5.0. Раздел requestSendPayment описывает планируемый API, наличие и контракт которого нужно проверить в установленной версии. Сквозная проверка с реальным платежом в рамках подготовки документа не проводилась.

2. Подключить приём оплаты

Шаг 1. Развернуть сервис

В каталоге DashPayBox соберите Docker-образ:

```
docker build -t dashpaybox:local .
```

Создайте файл `dashpaybox.env`, подставив адрес testnet и три независимых случайных секрета:

```
NETWORK=testnet
OWNER_STORAGE_ADDRESS=<адрес продавца в testnet>
AUTH_SECRET=<секрет API>
KEYS_ENCRYPTION_SECRET=<секрет шифрования ключей>
CALLBACK_URL=https://your-site.com/webhooks/dash
CALLBACK_SECRET=<секрет подписи webhook>
```

Например, каждый секрет можно получить командой `openssl rand -hex 32`. Храните файл вне репозитория с ограниченным доступом.

```
docker run -d --name dashpaybox --restart unless-stopped \
  --env-file dashpaybox.env \
  -p 127.0.0.1:8090:8090 \
  -v dashpaybox-data:/app/data dashpaybox:local
```

При таком запуске backend на том же хосте обращается к `http://127.0.0.1:8090`. Для backend в другом контейнере используйте общую Docker-сеть и адрес сервиса. Сохраняйте volume и секрет шифрования: они нужны для доступа к одноразовым ключам. DAPI-узлы обнаруживаются автоматически; `DAPI_SEEDS` позволяет задать доверенные узлы явно.

Шаг 2. Создать счёт из backend

```
const res = await fetch('http://127.0.0.1:8090/intents', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-Dash-Auth': process.env.DASH_AUTH_SECRET,
  },
  body: JSON.stringify({
    order_id: 'order-123',
    amount_minor: 1500, // 15 USD в центах
    currency: 'USD',
    instant_send: true,
    min_confirmations: 1,
  }),
});
if (!res.ok) throw new Error(`DashPayBox: ${res.status}`);
const invoice = await res.json();
```

Сумму определяет backend по данным заказа. Сохраните связь `order_id ↔ intent_id`. Каждый POST создаёт новый счёт: повторный запрос с тем же `order_id` не является идемпотентным. Защитите создание счёта от повторных кликов и повторов на стороне магазина.

3. Подтвердить оплату

Шаг 3. Показать счёт покупателю

Ответ `POST /intents` содержит `intent_id`, `address`, `duffs`, `amount`, `rate`, `rate_source`, `uri` и `expires_at`.

- Покажите адрес и точную сумму `amount` в DASH. При наличии метода предложите кнопку «Оплатить через расширение» (раздел 4).
- Сформируйте QR-код из `uri` вида `dash:ADDRESS?amount=...` и ссылку «Открыть кошелёк» с этим URI.
- Покажите срок оплаты: по умолчанию 900 секунд. Покупатель может платить Dash-кошельком без установки расширения.
- Frontend запрашивает состояние заказа у вашего backend. Для сверки backend использует `GET /intents/:id` с `X-Dash-Auth`.

Шаг 4. Проверить webhook

На `CALLBACK_URL` приходит POST. Проверяйте подпись по исходным байтам тела до JSON-разбора. Пример функции для Node.js:

```
import { createHmac, timingSafeEqual } from 'node:crypto';

function verifyWebhook(rawBody, signature, secret) {
  if (typeof signature !== 'string' ||
      !/^[0-9a-f]{64}$/i.test(signature)) return false;
  const expected = createHmac('sha256', secret)
    .update(rawBody).digest();
  const received = Buffer.from(signature, 'hex');
  return expected.length === received.length &&
    timingSafeEqual(expected, received);
}
// rawBody: Buffer из HTTP-запроса
// signature: заголовок X-Dash-Signature
// secret: значение CALLBACK_SECRET
```

event	Обработка
confirmed	Заказ оплачен. Переплата также считается оплатой.
mismatch	Недоплата: автоматическую выдачу не выполнять.
expired	Срок счёта истёк без оплаты.
late	Поздняя оплата: автоматическая выдача не выполняется, решение о возврате принимается вручную.

После проверки подписи сопоставьте `intent_id` и `order_id` с сохранённым счётом. В одной транзакции БД сохраните событие с уникальным ключом `(intent_id, event)` и измените заказ. Повтор события — без повторной выдачи. Отвечайте 2xx после надёжного сохранения; сервис повторяет неуспешную доставку.

AUTH_SECRET и CALLBACK_SECRET не передаются в браузер. Средства консолидируются на OWNER_STORAGE_ADDRESS; у sweep есть комиссия и минимальный порог. Автоматические возвраты и автосписания по подписке не реализованы.

CHECKOUT · ОПЛАТА ЧЕРЕЗ РАСШИРЕНИЕ

4. Вызвать requestSendPayment

> **Планируемый API.** Пример ниже основан на предлагаемом контракте. Перед подключением проверьте наличие метода в релизе расширения и окончательный контракт. В этом сценарии сайт вызывает один платёжный метод; необходимые разрешения и подтверждение обрабатывает расширение. Наличие метода не гарантирует поддержку всех источников средств.

DashPayBox возвращает Core-адрес счёта. По нажатию пользователя сайт передаёт его и точную сумму в расширение. Пользователь видит получателя, сумму и комиссии и подтверждает платёж внутри кошелька.

```
// invoice получен от вашего backend после создания счёта.
// Вызывайте из обработчика кнопки пользователя.
async function payWithExtension(invoice) {
  const extension = window.dashPlatformExtension;
  if (typeof extension?.requestSendPayment !== 'function') {
    throw new Error('Используйте QR-код или dash: ссылку');
  }
  if (Date.now() >= Date.parse(invoice.expires_at)) {
    throw new Error('Срок счёта истёк — запросите новый счёт');
  }

  const result = await extension.requestSendPayment({
    address: invoice.address,
    amountDuffs: String(invoice.duffs),
  });
  // Результат означает выполнение этапа операции в кошельке.
  // Статус заказа получайте от своего backend.
  return result;
}
```

`amountDuffs` — целочисленная строка: 1 DASH = 100 000 000 duffs. Предлагаемый контракт задаёт сумму получателю, комиссии оплачиваются сверху. Не пересчитывайте её из округлённого отображения DASH.

Сеть, источники и результат

Сайт не передаёт сеть и не переключает её запросом оплаты. Пользователь выбирает в расширении сеть, совпадающую с сетью магазина и DashPayBox. Расширение проверяет адрес и контекст операции. Для первого релиза ожидается Core → Core; оплата с Platform и Orchard зависит от поддержки соответствующих маршрутов в релизе.

После добавления выбора источников сайт по-прежнему передаёт только адрес и сумму. Расширение само определяет маршрут. Формат результата пока не закреплён: возможны Core txid, хеш State Transition и несколько идентификаторов для составного перевода. Успешный withdrawal в Platform ещё не равен поступлению в Core.

Состояния checkout

Пока запрос выполняется, блокируйте повторное нажатие. После результата показывайте «Ожидаем подтверждения оплаты». При отказе или ошибке объясните состояние; при таймауте сначала проверьте заказ через backend, не запускайте платёж автоматически повторно. Заказ выдавайте только после проверенного webhook confirmed.

QR-код, адрес и dash: URI остаются доступны для других кошельков и версий расширения без метода. Для платежей из Platform/Orchard учитывайте длительность вывода относительно expires_at; не обещайте мгновенное зачисление. Сеть магазина должна быть явно указана на checkout; расширение проверяет соответствие адреса текущей сети кошелька.

ИНТЕГРАЦИЯ · ПОЛНЫЙ СЦЕНАРИЙ

5. Связать компоненты

Backend магазина

1. По запросу авторизованного покупателя найдите заказ и определите сумму из своей базы.
2. Создайте счёт через POST /intents в DashPayBox. Сохраните intent_id вместе с заказом и верните frontend адрес, duffs, uri и expires_at.
3. На webhook проверьте HMAC и принадлежность счёта заказу. По confirmed атомарно отметьте заказ оплаченным и сохраните событие для защиты от повторной выдачи.
4. Предоставьте frontend свой endpoint статуса заказа. Не передавайте браузеру секрет доступа к DashPayBox.

Frontend магазина

```
// Пример endpoints магазина — их реализует ваш backend.
const invoiceResponse = await fetch(
  '/api/orders/order-123/dash-invoice', { method: 'POST' },
);
if (!invoiceResponse.ok) throw new Error('Не удалось создать счёт');
const invoice = await invoiceResponse.json();

// Сохраните invoice и покажите сумму, срок оплаты и кнопку.
// По нажатию кнопки:
const extension = window.dashPlatformExtension;
if (typeof extension?.requestSendPayment === 'function') {
  const result = await extension.requestSendPayment({
    address: invoice.address,
    amountDuffs: String(invoice.duffs),
  });
  // Покажите «Ожидаем подтверждения оплаты».
  // result не используется как доказательство оплаты заказа.
} else {
  // Покажите QR-код из invoice.uri и адрес для копирования.
}

// Запрашивайте статус у backend магазина:
// GET /api/orders/order-123/status
// Оплачено — только когда backend обработал confirmed.
```

Пример иллюстрирует поток; обработку ошибок, блокировку повторного нажатия и проверку expires_at добавьте в UI, как описано в разделе 4. Endpoints магазина должны проверять права покупателя на заказ.

Что делает покупатель

Устанавливает расширение, выбирает нужную сеть и пополняет кошелёк. На checkout нажимает «Оплатить через Dash Platform Extension», проверяет сумму и комиссию в кошельке и подтверждает. Выбор доступного источника средств и создание транзакций выполняет расширение.

> **Граница интеграции:** магазин выставляет счёт и ждёт подтверждение, расширение выполняет перевод, DashPayBox отслеживает поступление. Создавать транзакции через SDK, управлять nonce, ключами или State Transition разработчику магазина не нужно.

Исходники: github.com/lexxxell/dashpaybox · github.com/pshenmic/dash-platform-extension
Примеры backend и webhook: dashpaybox/examples/. Раздел requestSendPayment описывает планируемый контракт, который нужно сверить с выпущенной версией расширения.